

Software Requirements Engineering

Topics that will cover:

- Requirement Types: Functional & Non-functional
- Requirement Elicitation Techniques
- Requirement Analysis
- SRS (Software Requirement Specification)
- Use Case Diagrams

What is Software Requirements Engineering?

Software Requirements Engineering is all about understanding **what the software should do** and **how it should behave** before actually building it.

It includes:

- Collecting user needs
- Analyzing them
- Writing them clearly
- Making sure the software matches these needs later

Think of it like **asking a customer what kind of cake they want before baking it**. If you guess wrong, you'll waste time and ingredients!

Types of Requirements

There are two main types of requirements:

1. **Functional Requirements**
2. **Non-Functional Requirements**

1. Functional Requirements (What the system should do)

Definition:

Functional requirements describe the **core functions, features, and behaviors** the system must perform.

They answer the question:

- *"What should the software do?"*

These are the **visible parts** of the system that the user interacts with.

Examples of Functional Requirements:

For an online shopping app:

- The user should be able to **register** and **log in**
 - The user can **add items to the cart**
 - The app should **process payments** using credit cards
 - Admin can **add or remove products**
 - Users can **track their order status**
- These features directly relate to **how the software behaves**.

Who defines functional requirements?

- **Clients** (what they want)
- **End-users** (what they will use)
- **System analysts** (who write them down)
- **Developers** (who use them to build the software)

Why are they important?

- Functional requirements are the **foundation** of the software.
- If they are wrong or missing, the software **won't do what it's supposed to**.
- They are used to write **test cases** during testing.

2. Non-Functional Requirements (How the system should behave)

Definition:

Non-functional requirements describe the **quality** of the system — how fast, how secure, how easy to use, how reliable it is.

They answer the question:

- *"How should the system perform its tasks?"*

These are often **not visible** to users, but they affect **user experience, performance, and satisfaction**.

Examples of Non-Functional Requirements

For the same online shopping app:

- The app should load within **2 seconds**
- It should handle **1,000 users at the same time** (performance)
- It must follow **security standards** and use **data encryption**
- The app should be available **99.9% of the time** (availability)
- The app should work on **mobile, tablet, and desktop** (usability)

➤ These define the “**how well**” part of the system.

Common Categories of Non-Functional Requirements

Type	Meaning
Performance	Speed and responsiveness of the system
Security	Protection of data and system access
Usability	How easy the system is to use
Reliability	How often the system works without failure
Scalability	Can the system grow and handle more users?
Maintainability	How easy it is to update or fix the system
Portability	Can it run on different devices or OSs?

Why non-functional requirements matter?

Even if the software has the right **features**, if it:

- Loads too slowly
- Crashes often
- Is hard to use

Then the user will still be **unhappy**.

So, non-functional requirements are just as important as functional ones.

Comparison Table: Functional vs. Non-Functional

Feature	Functional Requirements	Non-Functional Requirements
Focus	What the system should do	How the system should perform
User Interaction	Direct (features the user uses)	Indirect (quality and performance)
Examples	Login, signup, checkout	Speed, security, usability
Testable?	Yes (through functional testing)	Yes (through performance, usability testing)
Importance	Defines core functionality	Ensures user satisfaction and trust

Requirement Elicitation Techniques

What is Requirement Elicitation?

- **Requirement Elicitation** means **gathering the needs and expectations** of users, clients, and stakeholders before developing the software.
- Think of it like this:
- Before you build someone a house, you need to **ask what kind of house** they want. How many rooms? What style? Where will the kitchen go?
- If you don't ask properly — you might build the wrong thing.
- Just like that, in software, we ask detailed questions to **understand exactly what to build**.

Why Elicitation Is Important?

- Helps you understand **what the user really needs**
 - Avoids **costly mistakes** or rework later
 - Builds **trust** with the client
 - Ensures the final product is **useful and accepted**
- It sets a **clear foundation** for design, development, and testing.

Techniques to Elicit Requirements

1. Interviews:

One-on-one conversation with stakeholders (clients, users).

- Ask open-ended questions like:
 - “What problem are you facing?”
 - “What features would help you?”
- **Types:** Structured (fixed questions), Unstructured (open discussion), or Semi-structured (mix of both)
- **Best for:** Getting detailed answers directly from the person who will use the software.

Example: “Can you walk me through how you currently handle student attendance?”

2. Questionnaires / Surveys:

A list of questions sent to many users to gather feedback.

- Good for getting opinions from a **large group** of people.
- Answers may be multiple choice, short answers, ratings, etc.

Best for: Collecting feedback from a **large group of people**

Example: Google Form asking teachers: “Which features do you want in a school management app?”

3.Observation:

- Watch how users currently perform tasks or in their natural environment.
- Quietly observe how users work — without asking much.
- Helps you find things users **may forget to mention**.

Use when: Users don't know how to describe what they do, or do tasks by habit

Best for: Understanding real user behavior in the workplace.

Example: Sitting in a hospital and watching how nurses update patient records

4.Workshops / Focus Groups:

Group sessions with users, developers, analysts, and stakeholders or Bring stakeholders together in a meeting to discuss the system.

- Everyone discusses the project, gives input, and clarifies doubts.
- Brainstorming, role-playing, or whiteboard sessions may be used.

Use when: You want to reach **agreement quickly** from different people

Best for: Large projects with multiple people involved.

Example: A group of teachers, admin staff, and developers discussing a new school system

5.Brainstorming:

Quick idea-generation sessions with stakeholders or team members.

- Everyone shares ideas freely — even wild ones!
- Helps discover **creative features** and **hidden needs**

Best for: When you're starting a project and need to explore possibilities.

Use when: You need **creative ideas** or **explore solutions**

Example: “What features would make our fitness app stand out?”

6.Prototyping:

Create a **mock-up or rough version** of the system or build a **basic working model** (prototype) and show it to users.

- Helps users **visualize** what the system might look like
- **Use when:** Users are **unsure** about what they want
- Users give feedback after seeing it.
- Helps refine unclear or incomplete requirements.

Best for: Projects where users are unsure what they want until they “see” it.

Example: A clickable app design showing how screens will work

7.Document Analysis:

- Study existing documents like old software manuals, reports, or user guides.
- Helps you learn from **past systems** or **similar projects**.

Use when: There’s already a system or process in place

Best for: Updating or replacing an old system.

Example: Analyzing hospital forms to understand how patient data is recorded

9.Use Case Modeling / Scenarios:

Describe **real-life user situations** to understand system needs.

- **Use when:** You want to see how a user would **interact with the system**

Example: “A student wants to borrow a book. What steps will they take?”

Summary Table of Elicitation Techniques

Technique	Best For	Example
Interviews	In-depth understanding	Talking to a teacher about daily tasks
Questionnaires	Quick input from many people	Google Forms from students
Observation	Learning real user behavior	Watching how attendance is taken
Workshops	Group discussion and fast feedback	Meeting with school staff
Brainstorming	Creative ideas and early feature planning	Generating app ideas
Prototyping	Showing rough design for feedback	Clickable app demo
Document Analysis	Understanding current or old systems	Reading hospital records
Scenarios	Mapping out user actions	Describing how a student borrows books

Requirement Analysis:

What is Requirement Analysis?

Requirement Analysis is the process of:

- Studying the collected requirements carefully,
 - **Understanding them clearly,**
 - **Organizing and checking** them,
 - Making sure they are **complete, correct, and doable** before building the software.
- **In simple words:**
It's like organizing ingredients before cooking you check what's available, what's missing, what needs to be cleaned or cut, and whether everything makes sense for the recipe.

Why Requirement Analysis Is Important?

If requirement analysis is skipped or done poorly:

- Developers **misunderstand** what to build
- Features may be **missing**
- Time and money may be **wasted**
- The client may **not be satisfied**

So, this step helps you build the **right software, the right way**.

What Happens During Requirement Analysis?

The **main steps** involved in requirement analysis:

1 Classify Requirements

Separate the requirements into different categories:

- **Functional requirements** → What the system should do
e.g., "Allow user to reset password"
- **Non-functional requirements** → How the system should behave
e.g., "Page should load within 2 seconds"
- **User requirements** → What the end-user needs
e.g., "I want a mobile app to track calories"
- **System requirements** → What the system must support internally
e.g., "The system must store user data in a secure database"

2 Check for Completeness

Make sure:

- Nothing important is missing
- Every user action is covered
- All conditions are considered

Example: If the user can “sign up” — can they also “reset their password”? Can they delete their account?

3 Check for Consistency

Make sure there are **no conflicts or contradictions**.

Example:

- Requirement A says: "Only admin can delete users"
- Requirement B says: "Users can delete their own accounts"
That's a conflict! It needs to be resolved.

4 Check for Feasibility

Ask: Can we actually **build this** with current tools, time, and skills?

Example:

If a client says “I want an app that works offline, online, and sends real-time alerts using AI”
→ You check if it's possible with the team and budget you have.

5 Prioritize Requirements

Not everything needs to be built first.

You divide requirements into:

- Must-have
- Nice-to-have
- Future improvements

Example: In a library app:

- Must-have: Search and borrow books
- Nice-to-have: Book reviews
- Future: Voice search

6 Modeling (Optional but Helpful)

Use diagrams or models to better **understand and communicate** the system.

Common tools:

- **Use Case Diagrams**
- **Data Flow Diagrams (DFD)**
- **Entity-Relationship Diagrams (ERD)**

These help visualize how data flows or how users interact with the system.

7 Validate with the Client

Once the analysis is done:

- Share it with the client or stakeholders
- Ask: “Is this what you meant?”
- Get **approval** before starting development

This prevents **costly changes** later.

SRS – Software Requirement Specification

What is an SRS?

An **SRS** is a detailed document that clearly defines **what the software should do** and **how it should behave** *before* development starts.

In simple words:

- It's like a **blueprint** for building software just like how an architect creates a design plan before building a house.
- Everything the developer needs to know is written down in the SRS so there's **no confusion later**.

Why is an SRS Important?

- Ensures **clear understanding** between clients and developers
- Avoids **misunderstandings and mistakes**
- Helps **testers** to verify if the software works as expected
- Acts as a **legal agreement or contract**
- Guides the **project team** step by step

Structure of an SRS:

1. **Introduction**
2. **Overall Description**
3. **Specific Requirements**
 - 3.1 Functional Requirements
 - 3.2 Non-Functional Requirements
 - 3.3 External Interfaces
4. **Use Case Diagrams (optional)**
5. **Glossary / Definitions**

What Should an SRS Contain?

A good SRS typically includes the following main sections:

1 Introduction

- Explains what the software is all about.

Includes:

- Project name
- Purpose of the software
- Who will use it (users/stakeholders)
- What it will and won't do (scope)

Example:

"This is a library management system for ABC School to manage book records, borrowing, and returns."

2 Overall Description

- Gives a general idea of how the system works.

Includes:

- Product functions (summary of major features)
- Users and their roles (e.g., student, librarian)
- Assumptions and dependencies (e.g., must have internet, or will run on Android)

Example:

- Students can search for and reserve books.
- Librarians can manage book inventory.

3 Specific Requirements

- This is the **core part** of the SRS. It lists **every requirement** in detail.

a) Functional Requirements

- What the system should do
- Features and actions of the software

Example:

- FR-1: The system shall allow the user to log in using email and password.
- FR-2: The librarian can add, update, and delete book records.

b) Non-Functional Requirements

- How the system should behave
- Quality factors like performance, speed, security

Example:

- NFR-1: The system shall load all pages within 3 seconds.
- NFR-2: The system must be available 24/7 with 99.9% uptime.

c) External Interface Requirements

- Describes how the system will interact with:
 - Users (UI)
 - Other systems (APIs)
 - Hardware or software (e.g., printer)

Example:

- The user interface shall be mobile-responsive.
- The system will connect to the college's ID verification API.

4 Use Case Diagrams or Models (Optional)

- Helps **visualize** how different users interact with the system.

Example Use Case:

- Student → Search book
- Librarian → Issue book

These are shown with simple diagrams (UML) to improve understanding.

5 Glossary / Definitions (Optional)

- Defines technical terms or short forms used in the SRS to make it easier for non-technical stakeholders to understand.

Example:

- API = Application Programming Interface
- UI = User Interface

How to Write Functional & Non-Functional Requirements in an SRS?

What is an SRS?

An **SRS** (Software Requirement Specification) is a **formal document** that clearly defines:

- What the software **should do** (functional requirements)
- How well it should do it (non-functional requirements)
- System behavior, constraints, assumptions, and more

Think of an SRS as a **contract** between the client and the development team. It avoids confusion, saves time, and ensures both sides understand the project clearly.

Structure of an SRS:

1.Introduction

2.Overall Description

3.Specific Requirements

- 3.1 Functional Requirements
- 3.2 Non-Functional Requirements
- 3.3 External Interfaces

4.Use Case Diagrams (optional)

5.Glossary / Definitions

Focus on **Section 3: Specific Requirements**, where we write functional and non-functional requirements.

How to Write Functional Requirements in SRS?

Format:

FR-x: *[Requirement Description]*

Where x is the number of the requirement (FR-1, FR-2, ...)

Example Functional Requirements (for an online bookstore):

FR-1: The system shall allow users to register with their name, email, and password.

FR-2: The system shall allow users to log in using their email and password.

FR-3: The system shall display a list of available books with title, author, and price.

FR-4: The system shall allow users to add books to a shopping cart.

FR-5: The system shall process payments via credit/debit cards and generate a receipt.

These are **actions or functions** the system must perform.

How to Write Non-Functional Requirements in SRS

Format:

NFR-x: *[Requirement Description]*

Where x is the number of the requirement (NFR-1, NFR-2, ...)

Example Non-Functional Requirements (same system):

- **NFR-1:** The system shall load all pages within 2 seconds under normal internet conditions. (*Performance*)
 - **NFR-2:** The system shall be available 24/7 with 99.9% uptime. (*Availability*)
 - **NFR-3:** The system shall use HTTPS for all transactions to ensure secure data transfer. (*Security*)
 - **NFR-4:** The user interface shall be responsive and work on desktops, tablets, and smartphones. (*Usability/Portability*)
 - **NFR-5:** The system shall support at least 5000 concurrent users without performance degradation. (*Scalability*)
- ✓ These describe **how well** the software should perform, not what it does.

SRS Template

Project Name: _____

Version: _____

Date: _____

Prepared by: _____

1. Introduction

1.1 Purpose

→ Briefly describe what the software will do.

1.2 Scope

→ Define what is included and what is NOT included.

1.3 Intended Audience

→ Who will use or read this document (developers, testers, clients, etc.)

1.4 Definitions & Acronyms

→ API = Application Programming Interface, UI = User Interface, etc.

2. Overall Description

2.1 Product Perspective

→ Is it a new system or an upgrade? Will it connect with other systems?

2.2 Product Features (Overview)

→ List major features (e.g., login, dashboard, analytics)

2.3 User Classes and Characteristics

→ Who are the users (admin, guest, customer)?

@Hira 2415 Student at Zamindar College, Gujrat

2.4 Operating Environment

→ Android, iOS, Web browser, Windows?

2.5 Assumptions and Dependencies

→ e.g., Requires internet connection, needs Google Maps API

3. Specific Requirements

3.1 Functional Requirements

- ✓ **FR-1:** The system shall allow users to register using email and password.
- ✓ **FR-2:** The system shall allow users to log in and access their dashboard.
- ✓ **FR-3:** The admin shall be able to add, delete, and update user data.
- ✓ **FR-4:** The user shall be able to upload a profile picture.
- ✓ **FR-5:** The system shall log the user's last login time.

3.2 Non-Functional Requirements

- ✓ **NFR-1:** The system shall respond to any request within 2 seconds.
- ✓ **NFR-2:** The UI shall be mobile responsive and user-friendly.
- ✓ **NFR-3:** The system shall support up to 10,000 concurrent users.
- ✓ **NFR-4:** Passwords shall be encrypted using SHA-256.
- ✓ **NFR-5:** The system shall maintain 99.9% uptime availability

4. External Interface Requirements

4.1 User Interface

→ Describe the layout of key screens (login, dashboard, etc.)

4.2 Hardware Interfaces

→ Any hardware? Barcode scanner, fitness watch?

4.3 Software Interfaces

→ API integration? Database type?

4.4 Communication Interfaces

→ e.g., Internet, Bluetooth, Wi-Fi support?

5. Use Case Diagrams (Optional)

→ Include a diagram showing actors and the use cases they interact with.

6. Glossary

→ Define any special terms used in this document.

Use Case Diagrams:

What is a Use Case Diagram?

A **Use Case Diagram** is a **visual diagram** that shows:

- Who will use the system (users called *actors*)
- What those users can do in the system (called *use cases*)

In simple words:

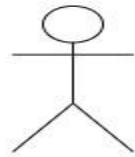
- A Use Case Diagram is like a **map** of your software that shows **who does what**.
- It helps everyone — developers, clients, testers — understand:
- Who interacts with the software
- What actions the software must support

Why Use Case Diagrams are Important

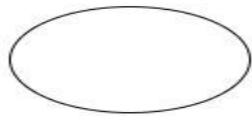
- Gives a **clear overview** of how users interact with the system
- Helps during **requirement gathering and validation**
- Used in **SRS documents**
- Improves **communication** between developers and clients

Main Components of a Use Case Diagram:

Component	Symbol	Description
Actor	 (Stick figure)	A person or system that interacts with your software
Use Case	 (Oval shape)	A specific action the system performs
System Boundary	 (Rectangle)	Shows what's inside and outside the system
Associations	 (Lines)	Connects actors to use cases
Relationships	 Arrows/Lines	Includes, extends, generalization



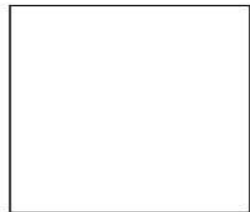
Actor



Use Case



Communication Link



Boundary Of System



Extends



Include



Generalization

Actors

- Can be a **human user** (e.g., Student, Admin)
- Can also be **another system** (e.g., Payment Gateway)

Use Cases

Each oval represents something the user **wants to do** with the system:

- Login
- Search book
- Add to cart
- Send message

System Boundary

This is the **rectangle** that contains all the use cases — representing the **software system** itself.

Relationships in Use Case Diagrams:

Relationship Type	Symbol	Meaning
Association	Line	Actor is connected to a use case
Include	Dotted arrow (→ with <>)	A use case always calls another use case
Extend	Dotted arrow (→ with <>)	A use case may optionally call another use case

Example 2: Use Case Diagram for a Fitness App

Actors:

- User
- Admin

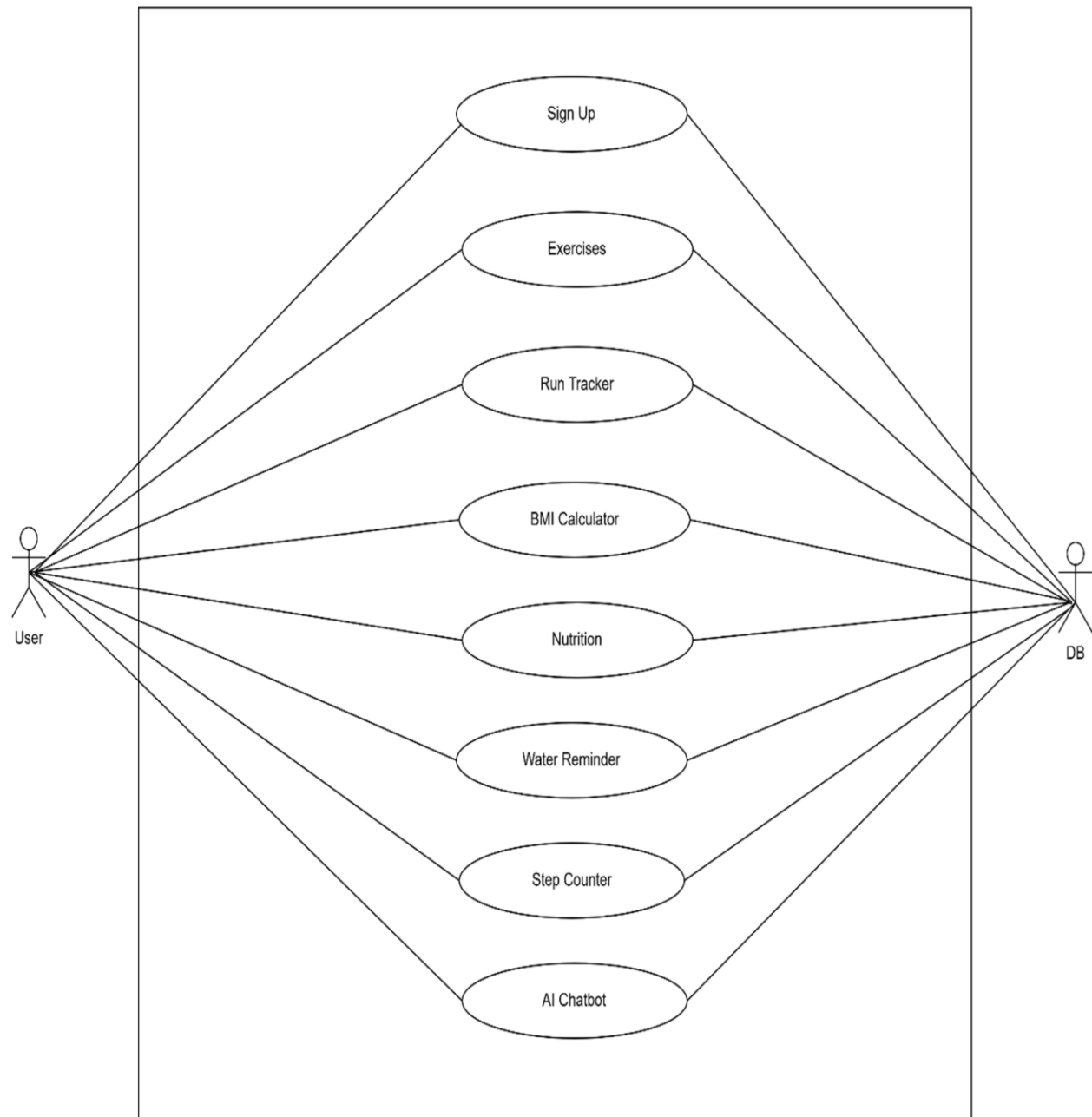
Use Cases:

- Track Steps
- Set Goal
- View Progress
- Send Notification
- Update User Profile

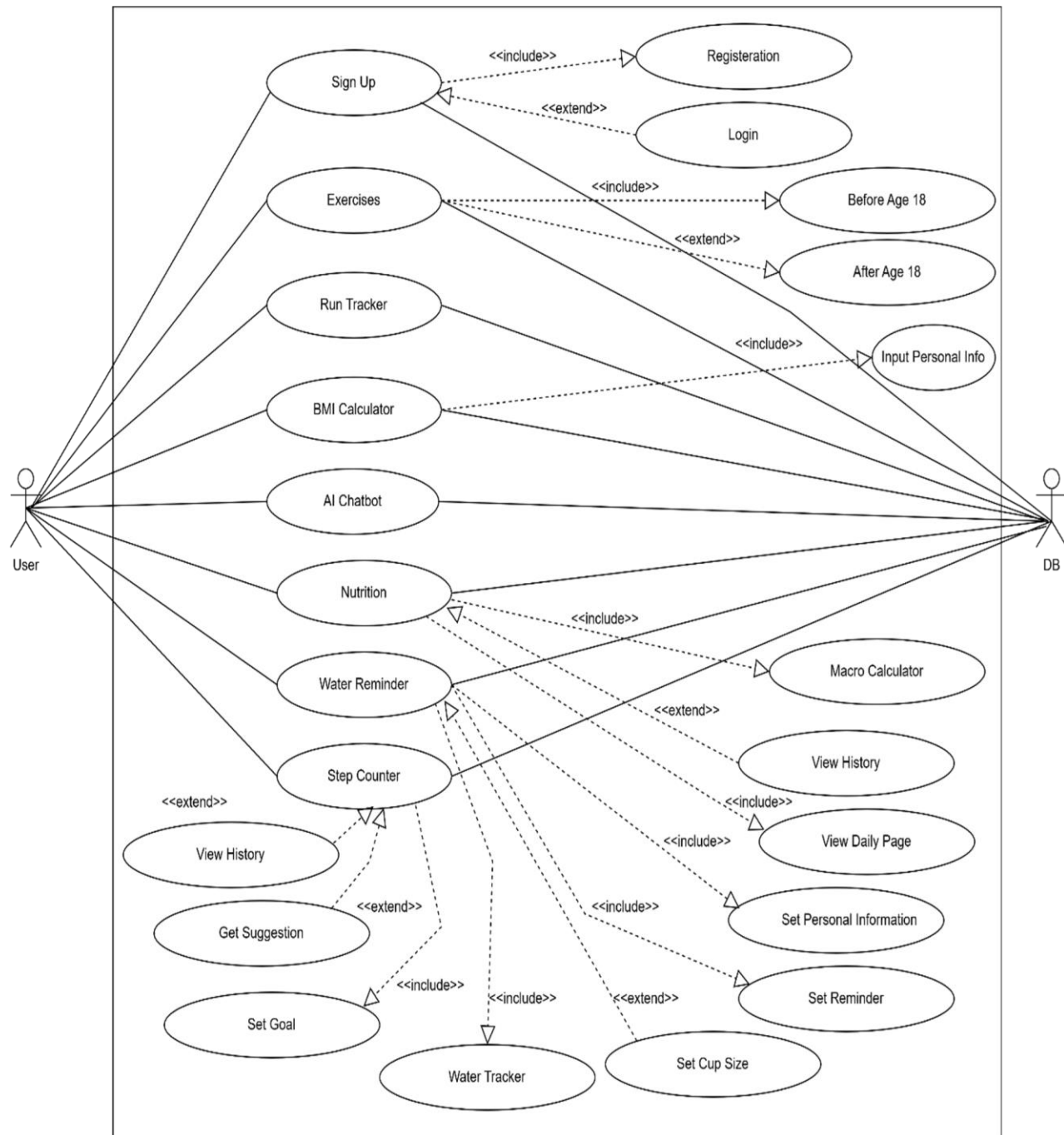
User can: Track Steps, Set Goal, View Progress, Update Profile

Admin can: Send Notification

Use Case Diagram:



Analysis Level Use Case Diagram



Types of Use Case diagram:

1. High-Level Use Case Diagram
2. Analysis level Use Case Diagram

1.High level Use Case Diagram:

Definition:

A **High-Level Use Case Diagram** gives a **quick overview** of the **main features** the system provides **without too much detail**.

Key Points:

- Focuses on **major functionalities only**
- Only **main actors** and **main use cases** shown
- Useful for **client presentations** or quick understanding

High-Level Use Case Diagram for FitTrack App

Actors:

- User
- Admin

Main Use Cases:

- Register/Login
- Track Steps
- Set Daily Goal
- View Progress
- Log Water Intake
- View Exercise Plan
- Get Nutrition Tips

2.Analysis-Level Use Case Diagram:

Definition:

An **Analysis-Level Use Case Diagram** shows **more detailed interactions** between users and the system. It is used during the **requirements analysis phase** to **capture all actions clearly**.

Key Points:

- **More detailed** than high-level
- May include:
 - **Sub-use cases**
 - <<include>> and <<extend>> relationships
 - Multiple diagrams (one per actor or feature)
- Helps developers and testers understand **exactly what's needed**

Analysis-Level Use Case Diagram for “Track Steps” Feature

Let's take one use case from above and **zoom in**: Track Steps

Actor:

- User

Detailed Use Cases:

- Start Step Counter
 - <<include>> Read Sensor Data
 - <<include>> Calculate Distance
 - <<include>> Calculate Calories Burned
 - <<extend>> Send Goal Notification (if goal is achieved)
-